CUSTOMER ENGAGEMENT & SERVICE ANALYTICS

Name : M VIJAY SIMHA REDDY
Date : 18 - 03 - 2026
Batch Name : .Net Full Stack React (MS Dynamics)
Instructor : Parth Shukla

Table of Contents :

SNo.	Contents	Page No.
1.	Problem Definition	2
2.	Project Objectives	2
3.	System Architecture	3
4.	Technology Stack	4
5.	Backend Architecture	4
6.	Frontend Architecture	4
5.	ER Diagram	5
6.	Backend Implementation	6
7.	Frontend Implementation	7
8.	SSRS Integration	8
9.	Reporting & Analytics (Power BI)	8
10.	Azure Integration	9
11.	Conclusion	10

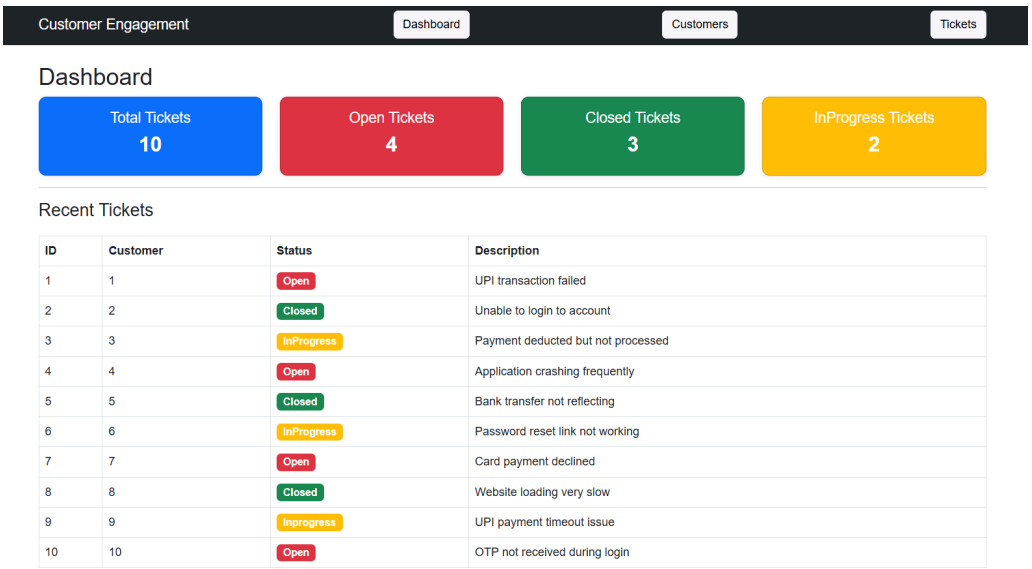
1. Problem Definition

ConnectPlus wants to modernize its Customer Engagement & Ticket Management System with a focus on service efficiency, transparency, and analytics-driven decision-making. Organizations handling customer service requests often face issues such as:

- Delayed ticket resolution
- Duplicate customer requests
- Lack of centralized ticket tracking
- Limited visibility into service performance
- Inconsistent reporting

These issues reduce operational efficiency and impact customer satisfaction.

User Interface :



ID	Customer	Status	Description
1	1	Open	UPI transaction failed
2	2	Closed	Unable to login to account
3	3	InProgress	Payment deducted but not processed
4	4	Open	Application crashing frequently
5	5	Closed	Bank transfer not reflecting
6	6	InProgress	Password reset link not working
7	7	Open	Card payment declined
8	8	Closed	Website loading very slow
9	9	Inprogress	UPI payment timeout issue
10	10	Open	OTP not received during login

2. Project Objectives

The primary objectives of the system are:

- Manage customer profiles and service tickets in a centralized system.
- Prevent duplicate tickets and invalid ticket workflow transitions.
- Provide a responsive UI for support agents.
- Generate service analytics and operational reports.
- Support scalable architecture for future integrations.

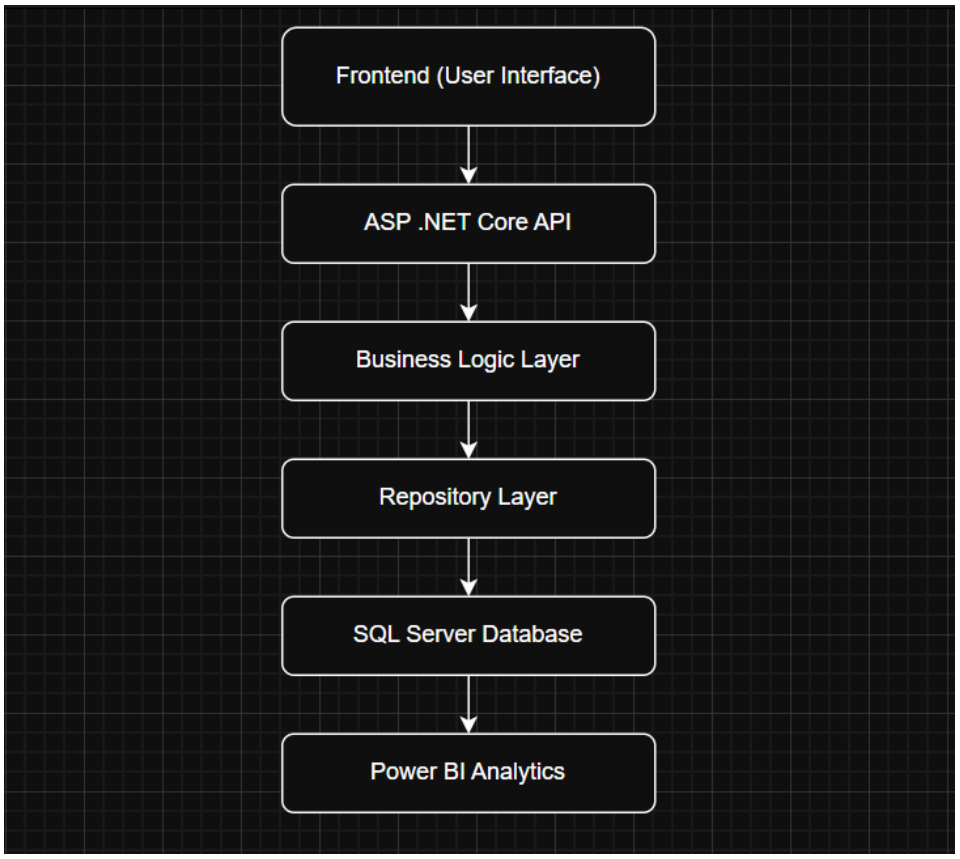
The system is designed to support both operational management and analytical decision-making.

3. System Architecture

The project follows a layered architecture. And it contains 3 Layers

1. Frontend Layer
   HTML, CSS, JavaScript, BootStrap, TypeScript (Provides the user interface for Customer Management Dashboard).
2. Backend Layer
   ASP .NET Core Web API, C#, OOP and SOLID Principles (handles business logic, API processing and validation).
3. Database Layer
   SQL Server Database (For storing data of customers and tickets).

Architecture Flow :



This architecture ensures separation of concerns and scalability.

4. Technology Stack

Layer	Technology
Frontend	HTML, CSS, JavaScript, BootStrap, TypeScript
Backend	ASP .NET Core Web API
Programming	C#
Database	Microsoft SQL Server
Analytics	Power BI
Cloud (conceptual)	Azure Active Directory

This stack supports scalable and maintainable enterprise applications.

5. Backend Architecture

The backend follows SOLID design principles.
Layers used :

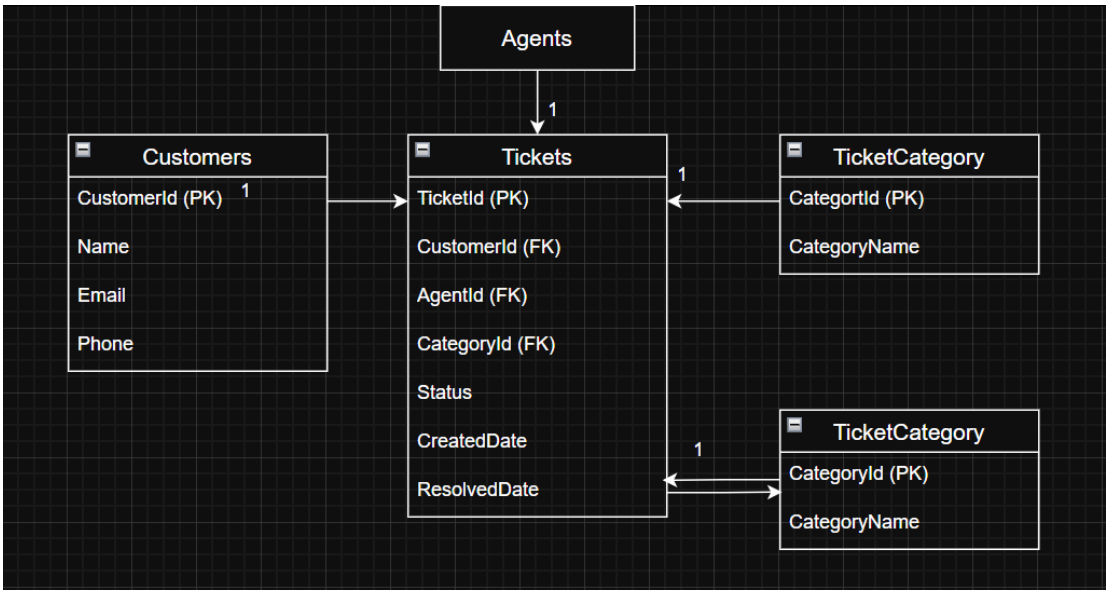
- Models → Represents database entities.
- Repository → Handles Database operations.
- Service → Contains business logic.
- Controllers → Expose REST APIs.
- Data → Manages database interaction using entity framework.
- Exceptions → Handles errors in the application.
- Interfaces → Defines the contracts of repository and service layers.

6. Frontend Architecture

The frontend provides a responsive UI for support agents.
Main components :

- Customer Management → Create, view and search Customers.
- Ticket Management → Create tickets, Update status, Assign agents and track ticket progress
- SPA (Single Page Application) → The application uses JavaScript DOM manipulation to update data dynamically without full page reloads.This improves performance and usability.

7. ER Diagram



The Entity-Relationship Diagram (ERD) represents the logical structure of the database and the relationships between tables.

Entities :

- Customers
- Agents
- Tickets
- TicketCategory

ERD Relationships :

- One customer can create multiple tickets.
- One agent can handle multiple tickets.
- One ticket category can have many tickets.

Foreign keys maintain referential integrity between related tables.

8. Backend Implementation

The backend of the Customer Engagement & Service Analytics Platform is implemented using ASP.NET Core Web API with C#.

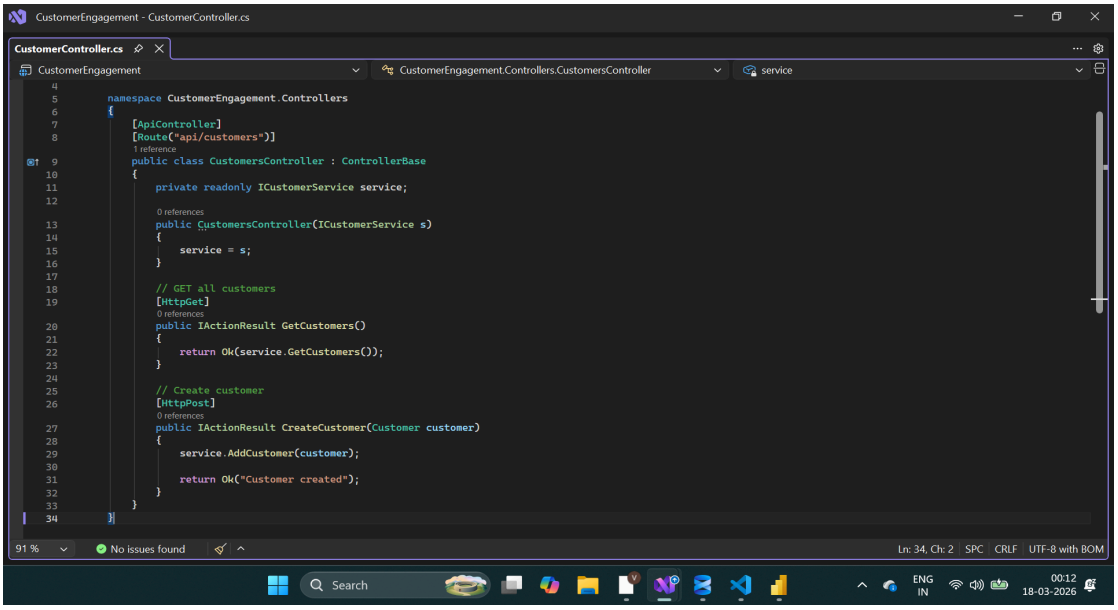
It follows a layered architecture that separates concerns across multiple components such as models, repositories, services, and controllers.

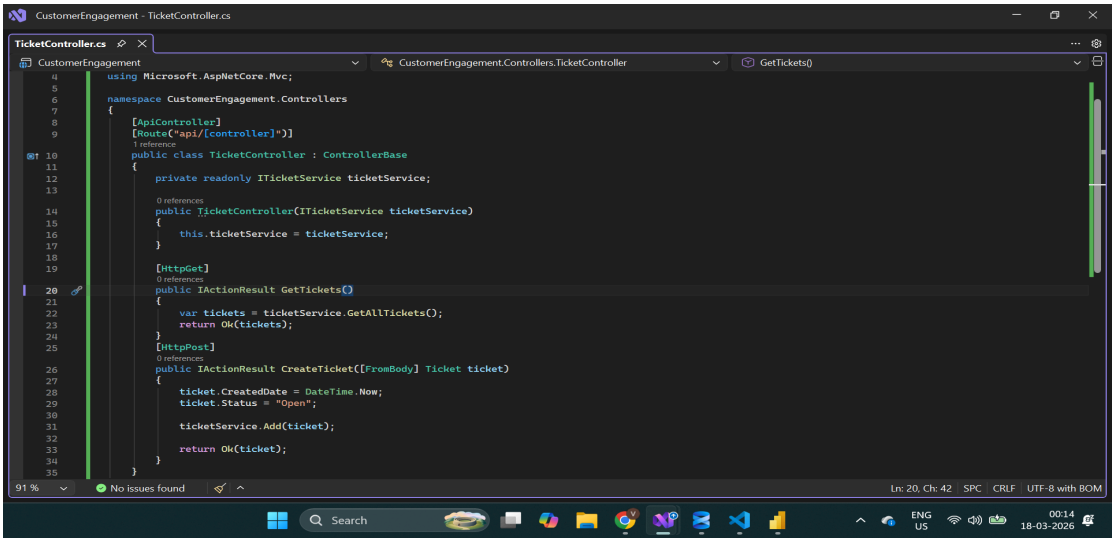
This architecture improves maintainability, scalability, and testability of the application.

The backend is responsible for:

- Managing customer and ticket data.
- Enforcing business rules.
- Handling database operations.
- Exposing REST APIs for the frontend.

Validation Logic:





9. Frontend Implementation

The frontend provides a user-friendly interface for support agents and supervisors to manage customer service operations.

The interface is built using :

- HTML
- CSS
- JavaScript
- BootStrap
- TypeScript

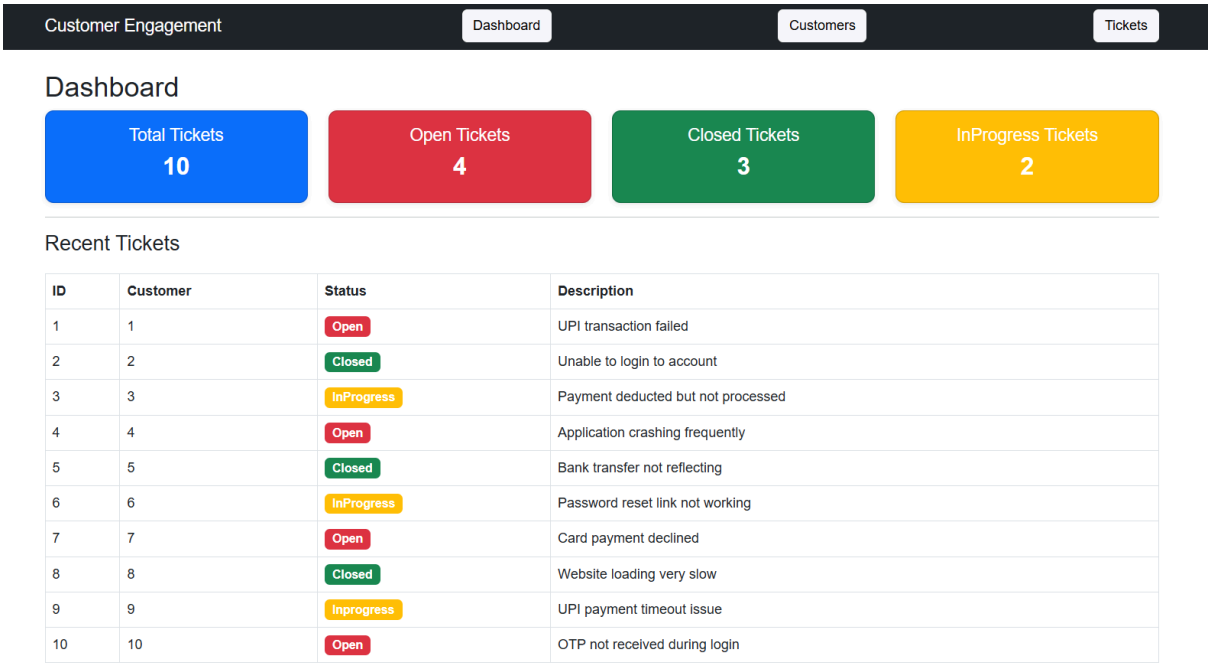
In the Customer Engagement & Service Analytics Platform, SPA concepts are used to ensure efficient interaction between the user interface and backend services.

The application uses JavaScript and TypeScript to dynamically fetch and update data from the backend APIs without refreshing the entire page. Instead of loading new pages, the system dynamically updates sections of the interface.

- Creating new tickets.
- Updating ticket status.
- Viewing customer records.

These operations are performed without refreshing the entire page.

Dashboard :

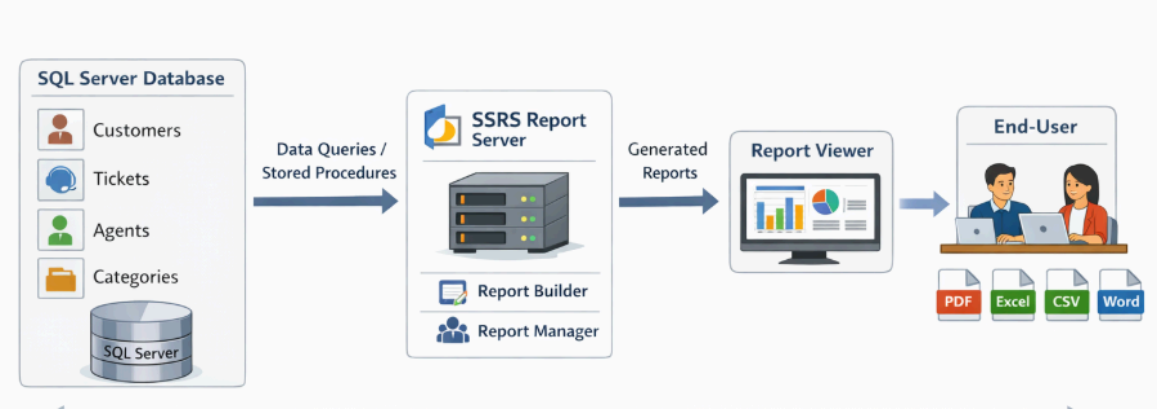


ID	Customer	Status	Description
1	1	Open	UPI transaction failed
2	2	Closed	Unable to login to account
3	3	InProgress	Payment deducted but not processed
4	4	Open	Application crashing frequently
5	5	Closed	Bank transfer not reflecting
6	6	InProgress	Password reset link not working
7	7	Open	Card payment declined
8	8	Closed	Website loading very slow
9	9	Inprogress	UPI payment timeout issue
10	10	Open	OTP not received during login

10. SSRS Integration

SQL Server Reporting Services (SSRS) is integrated into the Customer Engagement & Service Analytics Platform to generate structured operational reports. SSRS provides a platform for creating, managing, and delivering reports based on SQL Server data.

SSRS is used to generate reports that help customer support managers monitor ticket activity, service performance, and agent productivity. These reports provide a clear view of operational data and support decision-making.



11. Reporting & Analytics (Power BI)

Power BI is used in the Customer Engagement & Service Analytics Platform to create interactive dashboards and data visualizations that provide insights into customer service operations.

KPI Cards :

- Total Tickets.
- Open Tickets.
- Closed Tickets.
- Average Resolution Time.

These visuals provide a quick overview of system performance.

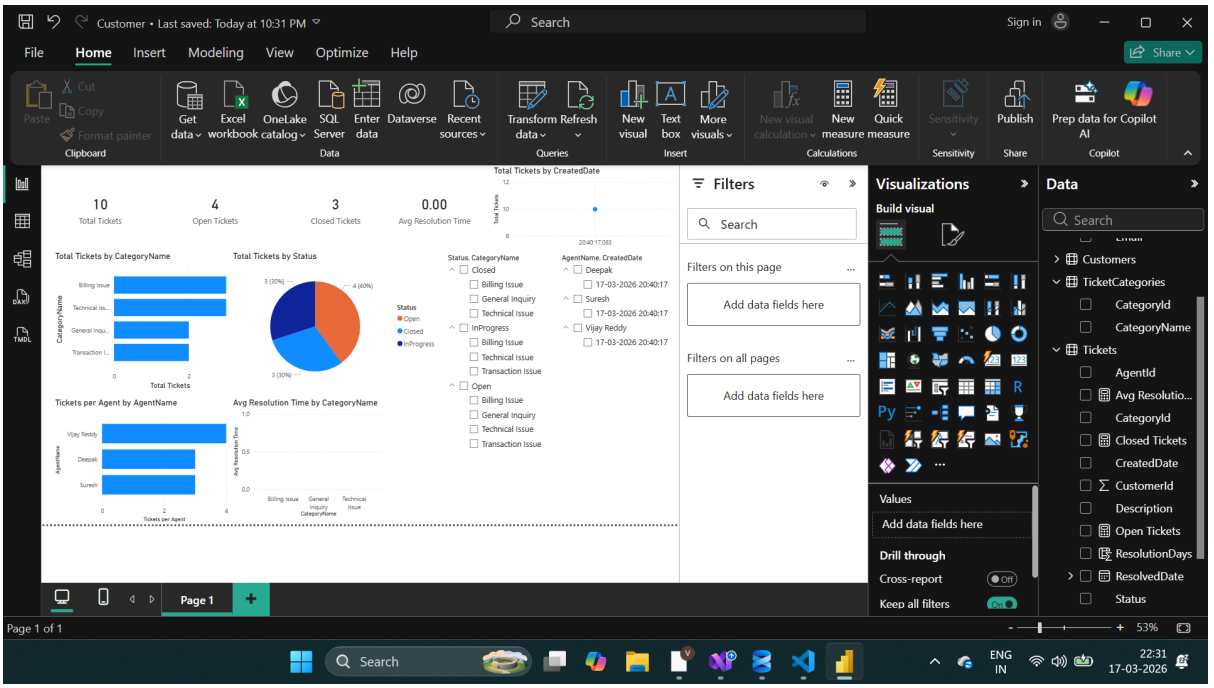
Pie Chart :

Shows the percentage of tickets based on their current status.

- Open
- InProgress
- Closed

This visualization helps identify service backlog and ticket distribution.

Power BI Dashboard design :

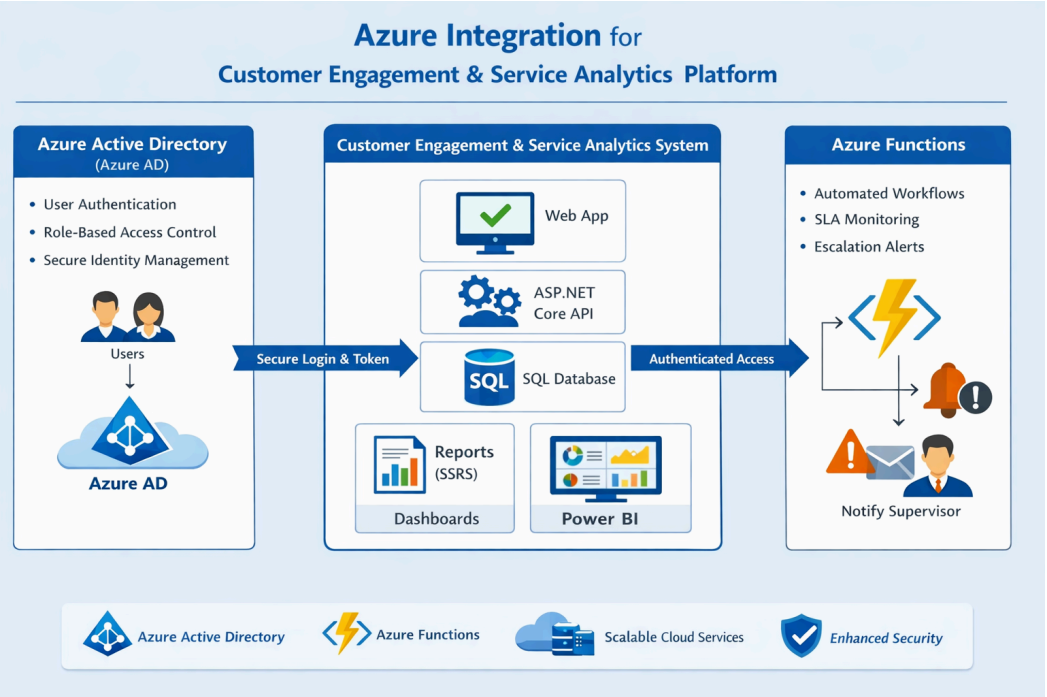


12. Azure Integration

Azure integration enhances the Customer Engagement & Service Analytics Platform by introducing secure authentication and scalable cloud-based services. Microsoft Azure provides a set of cloud tools that help improve application security, automation, and scalability.
In this project, Azure services are considered for secure identity management and automated service operations. These integrations ensure that the platform can be extended for enterprise-level deployment in the future.

Azure Active Directory (Azure AD) can be integrated into the system to provide secure authentication and role-based access control for users of the platform.

Azure Functions can be used to automate specific operations within the customer service system.In this project, Azure Functions can be used for automatic ticket escalation when Service Level Agreement (SLA) limits are exceeded.



13. Conclusion

The Customer Engagement & Service Analytics Platform is successfully designed and implemented to address common challenges faced by organizations in managing customer service requests. The system provides a centralized platform for managing customer profiles, service tickets, and support agent activities while ensuring data consistency and operational efficiency.

The project follows a structured architecture that includes a frontend interface, a backend REST API built using ASP.NET Core, and a relational SQL Server database. The backend architecture uses a layered design with models, repositories, services, and controllers, ensuring clean separation of concerns and maintainable code.

Overall, the project demonstrates the implementation of a scalable and modular customer support system that combines modern web development practices with data analytics capabilities. The platform enables organizations to improve customer service operations, monitor performance metrics, and gain valuable insights into service efficiency.